

Algorithms

Divide and Conquer

Dr. Mudassir Shabbir

LUMS University

March 9, 2026

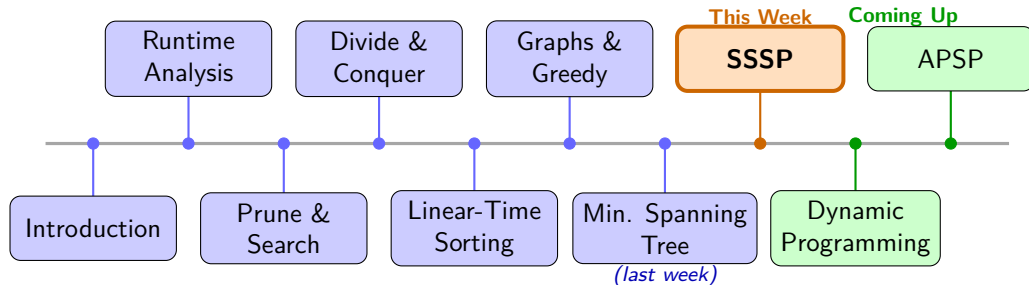


Announcements

- HW 3 is due on [Feb 09](#) at **11:55 PM**.
- HW 4 is out - no submission required.



Recap & What's Next



From Spanning Trees to Shortest Paths

What we just studied:

- MST finds edges to *connect all vertices*.
- Goal: minimize *total edge weight*.
- Prim's grows a tree by always adding the cheapest crossing edge.

A different question:

- What if we want the *shortest path* from a source s to every other vertex?
- Goal: minimize *weighted path length* from s .
- This is the **Single-Source Shortest Path** problem.



From Spanning Trees to Shortest Paths

What we just studied:

- MST finds edges to *connect all vertices*.
- Goal: minimize *total edge weight*.
- Prim's grows a tree by always adding the cheapest crossing edge.

A different question:

- What if we want the *shortest path* from a source s to every other vertex?
- Goal: minimize *weighted path length* from s .
- This is the **Single-Source Shortest Path** problem.



From Spanning Trees to Shortest Paths

What we just studied:

- MST finds edges to *connect all vertices*.
- Goal: minimize *total edge weight*.
- Prim's grows a tree by always adding the cheapest crossing edge.

A different question:

- What if we want the *shortest path* from a source s to every other vertex?
- Goal: minimize *weighted path length* from s .
- This is the **Single-Source Shortest Path** problem.

Real-world applications

GPS navigation, internet routing (BGP, OSPF), social networks (degrees of separation), flight connections, supply chain optimization.



The Shortest Path Problem

Given: A directed or undirected graph $G = (V, E, W)$ with **no-negative-weight-cycles**¹ and a source vertex $s \in V$.

¹no cycles with total weight < 0



The Shortest Path Problem

Given: A directed or undirected graph $G = (V, E, W)$ with **no-negative-weight-cycles**¹ and a source vertex $s \in V$.

Goal: Find the shortest path from s to every other vertex $v \in V$.

¹no cycles with total weight < 0



The Shortest Path Problem

Given: A directed or undirected graph $G = (V, E, W)$ with **no-negative-weight-cycles**¹ and a source vertex $s \in V$.

Goal: Find the shortest path from s to every other vertex $v \in V$.

The **distance** $d(v)$ is the minimum sum of edge weights on any path from s to v .

¹no cycles with total weight < 0

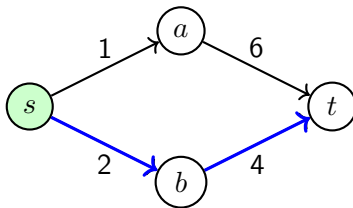


The Shortest Path Problem

Given: A directed or undirected graph $G = (V, E, W)$ with **no-negative-weight-cycles**¹ and a source vertex $s \in V$.

Goal: Find the shortest path from s to every other vertex $v \in V$.

The **distance** $d(v)$ is the minimum sum of edge weights on any path from s to v .



Shortest path: $s \rightarrow b \rightarrow t$ with distance $2 + 4 = 6$

¹no cycles with total weight < 0

Dijkstra's Algorithm

Maintain for each vertex v :

- $d[v]$: current best known distance from s to v .
(Initially ∞ for all $v \neq s$, and 0 for s .)
- $\text{parent}[v]$: predecessor on the shortest path.



Dijkstra's Algorithm

Maintain for each vertex v :

- $d[v]$: current best known distance from s to v .
(Initially ∞ for all $v \neq s$, and 0 for s .)
- $\text{parent}[v]$: predecessor on the shortest path.

A **min-priority queue** Q holds all unprocessed vertices keyed by $d[v]$.



Dijkstra's Algorithm

Maintain for each vertex v :

- $d[v]$: current best known distance from s to v .
(Initially ∞ for all $v \neq s$, and 0 for s .)
- $\text{parent}[v]$: predecessor on the shortest path.

A **min-priority queue** Q holds all unprocessed vertices keyed by $d[v]$.

Steps:

- 1 $d[s] \leftarrow 0$; $d[v] \leftarrow \infty$ for $v \neq s$.
- 2 Insert all vertices into Q .
- 3 While $Q \neq \emptyset$:
 - $u \leftarrow \text{ExtractMin}(Q)$
 - For each neighbor v of u :
if $d[u] + w(u, v) < d[v]$:
 - $d[v] \leftarrow d[u] + w(u, v)$
 - $\text{parent}[v] \leftarrow u$
 - $\text{DecreaseKey}(Q, v)$



Dijkstra's Algorithm

Maintain for each vertex v :

- $d[v]$: current best known distance from s to v .
(Initially ∞ for all $v \neq s$, and 0 for s .)
- $\text{parent}[v]$: predecessor on the shortest path.

A **min-priority queue** Q holds all unprocessed vertices keyed by $d[v]$.

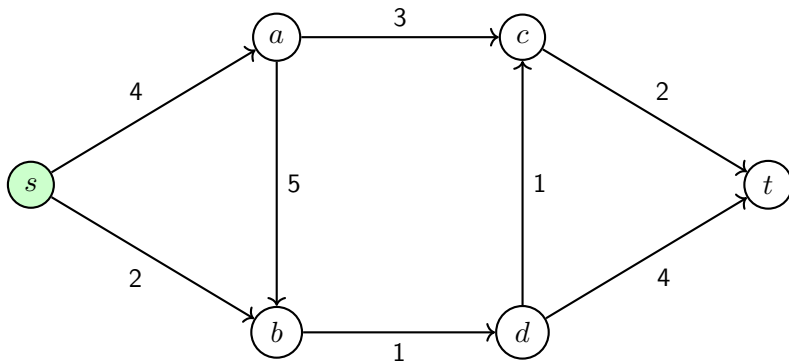
The step in blue is called **relaxing** edge (u, v) .

Steps:

- 1 $d[s] \leftarrow 0$; $d[v] \leftarrow \infty$ for $v \neq s$.
- 2 Insert all vertices into Q .
- 3 While $Q \neq \emptyset$:
 - $u \leftarrow \text{ExtractMin}(Q)$
 - For each neighbor v of u :
if $d[u] + w(u, v) < d[v]$:
 - $d[v] \leftarrow d[u] + w(u, v)$
 - $\text{parent}[v] \leftarrow u$
 - $\text{DecreaseKey}(Q, v)$

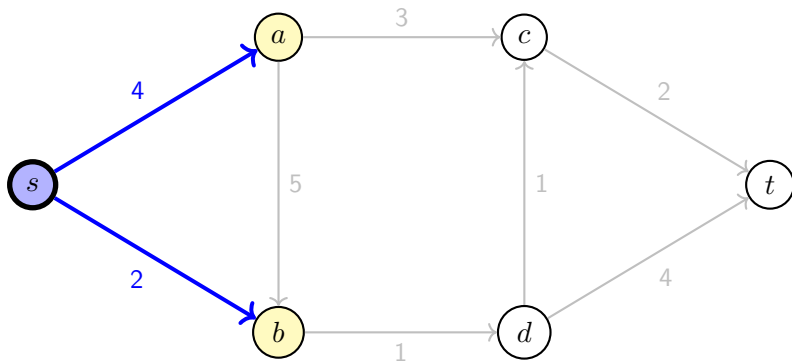


Dijkstra in Action: Example Graph



Starting from source s . Initial distances: $d[s] = 0$, all others ∞ .

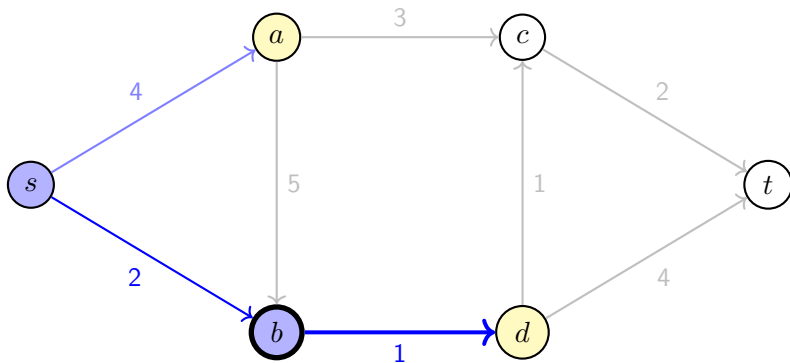
Step 1: Extract s (distance 0)



Vertex	s	a	b	c	d	t
$d[v]$	0	4	2	∞	∞	∞

Relax edges from s : update $d[a] = 4$ and $d[b] = 2$.

Step 2: Extract b (distance 2)

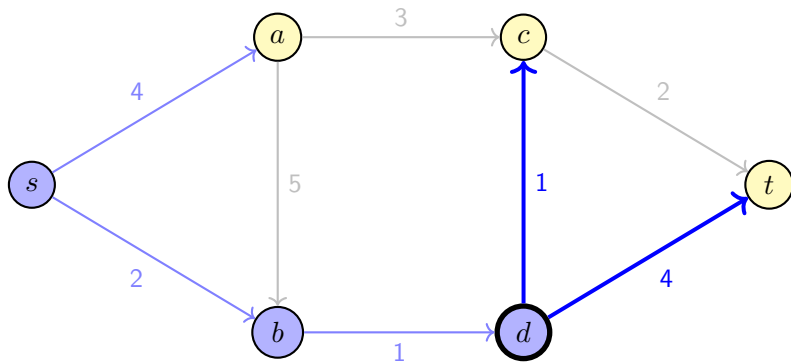


Vertex	s	a	b	c	d	t
$d[v]$	0	4	2	∞	3	∞

Relax edge $b \rightarrow d$: update $d[d] = 2 + 1 = 3$. (Note: $b \rightarrow a$ doesn't improve $d[a]$.)



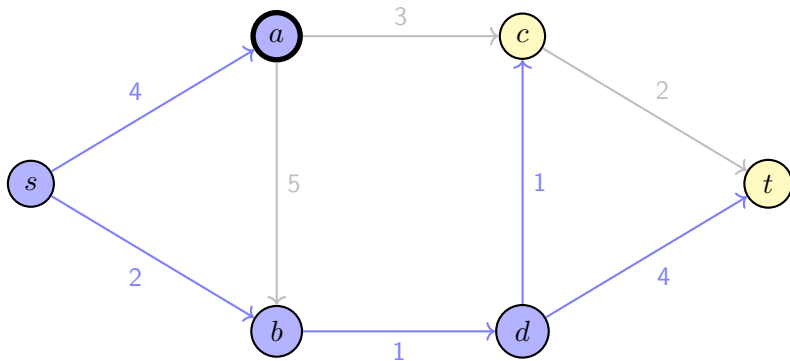
Step 3: Extract d (distance 3)



Vertex	s	a	b	c	d	t
$d[v]$	0	4	2	4	3	7

Relax edges from d : update $d[c] = 3 + 1 = 4$ and $d[t] = 3 + 4 = 7$.

Step 4: Extract a (distance 4)

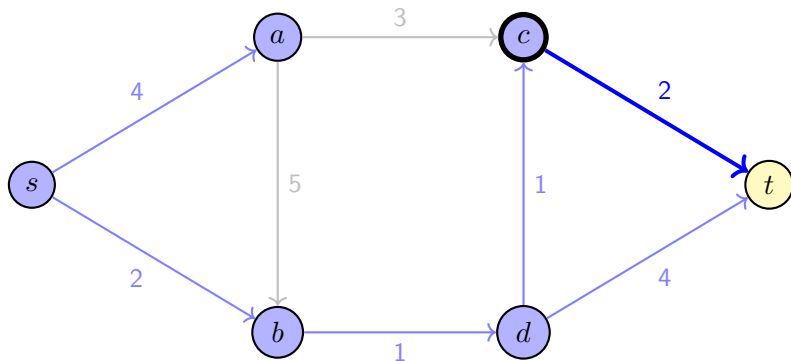


Vertex	s	a	b	c	d	t
$d[v]$	0	4	2	4	3	7

Relax edge $a \rightarrow c$: $4 + 3 = 7 \not< 4$, so no update. (Also $a \rightarrow b$ doesn't help.)



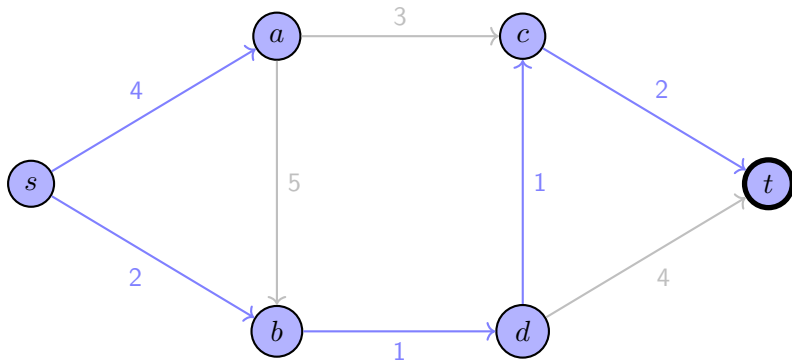
Step 5: Extract c (distance 4)



Vertex	s	a	b	c	d	t
$d[v]$	0	4	2	4	3	6

Relax edge $c \rightarrow t$: update $d[t] = 4 + 2 = 6$ (better than previous 7).

Step 6: Extract t (distance 6) — Done!



Vertex	s	a	b	c	d	t
$d[v]$	0	4	2	4	3	6

Final shortest distances from s : All vertices processed!



Dijkstra's Correctness: The Greedy Choice is Safe

Claim: When Dijkstra extracts vertex u from the priority queue, $d[u]$ is the true shortest distance from s to u .



Dijkstra's Correctness: The Greedy Choice is Safe

Claim: When Dijkstra extracts vertex u from the priority queue, $d[u]$ is the true shortest distance from s to u .

Proof sketch (by contradiction):



Dijkstra's Correctness: The Greedy Choice is Safe

Claim: When Dijkstra extracts vertex u from the priority queue, $d[u]$ is the true shortest distance from s to u .

Proof sketch (by contradiction):

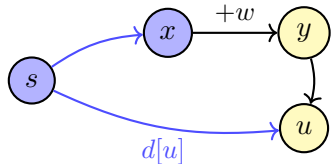
- Suppose not. Let u be the *first* vertex extracted where $d[u]$ is not optimal.
- Let P be a truly shorter path from s to u .
- Consider the first edge (x, y) on P where x has been extracted but y has not.
- Since all weights are non-negative:
$$d[y] \leq d[x] + w(x, y) \leq \text{length of } P < d[u]$$
- But u was extracted before y , so $d[u] \leq d[y]$. **Contradiction!**

Conclusion: Every extracted vertex has its final shortest distance. *Dijkstra is correct.*



Visual Proof: Why Non-Negative Weights Matter

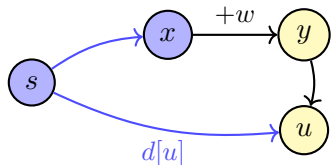
With non-negative weights:



Any path through unprocessed vertices adds positive weight, so can't be shorter than $d[u]$.

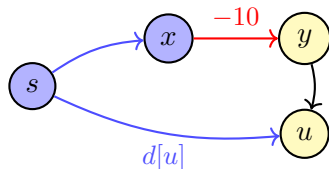
Visual Proof: Why Non-Negative Weights Matter

With non-negative weights:



Any path through unprocessed vertices adds positive weight, so can't be shorter than $d[u]$.

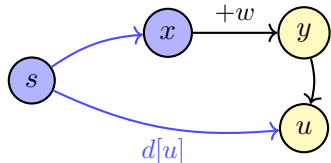
With negative weights:



A path through y might be shorter even though $d[u]$ looks final. **Dijkstra fails!**

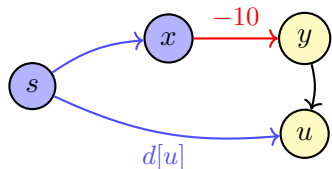
Visual Proof: Why Non-Negative Weights Matter

With non-negative weights:



Any path through unprocessed vertices adds positive weight, so can't be shorter than $d[u]$.

With negative weights:



A path through y might be shorter even though $d[u]$ looks final. **Dijkstra fails!**

Key Takeaway

Non-negative weights ensure that once a vertex is extracted, no future path can improve its distance. This is the heart of Dijkstra's correctness.

Dijkstra's Time Complexity

The main loop runs V times (one `ExtractMin` per vertex).

Each edge is relaxed at most once, triggering at most one `DecreaseKey`.



Dijkstra's Time Complexity

The main loop runs V times (one `ExtractMin` per vertex).

Each edge is relaxed at most once, triggering at most one `DecreaseKey`.

Simple array (unsorted list):

- `ExtractMin`: scan all entries.
- `DecreaseKey`: update one cell.

$O(V)$

$O(1)$



Dijkstra's Time Complexity

The main loop runs V times (one ExtractMin per vertex).

Each edge is relaxed at most once, triggering at most one DecreaseKey.

Simple array (unsorted list):

- ExtractMin: scan all entries.
- DecreaseKey: update one cell.
- Total: $V \cdot O(V) + E \cdot O(1)$
- *Good for dense graphs where $E \approx V^2$.*

$$O(V)$$

$$O(1)$$

$$= O(V^2)$$



Dijkstra's Time Complexity

The main loop runs V times (one ExtractMin per vertex).
Each edge is relaxed at most once, triggering at most one DecreaseKey.

Simple array (unsorted list):

- ExtractMin: scan all entries.
- DecreaseKey: update one cell.
- Total: $V \cdot O(V) + E \cdot O(1)$
- *Good for dense graphs where $E \approx V^2$.*

$$O(V)$$

$$O(1)$$

$$= O(V^2)$$

Binary heap:

- Both ExtractMin and DecreaseKey: $O(\log V)$.



Dijkstra's Time Complexity

The main loop runs V times (one `ExtractMin` per vertex).

Each edge is relaxed at most once, triggering at most one `DecreaseKey`.

Simple array (unsorted list):

- `ExtractMin`: scan all entries. $O(V)$
- `DecreaseKey`: update one cell. $O(1)$
- Total: $V \cdot O(V) + E \cdot O(1)$ $= O(V^2)$
- *Good for dense graphs where $E \approx V^2$.*

Binary heap:

- Both `ExtractMin` and `DecreaseKey`: $O(\log V)$.
- Total: $V \cdot O(\log V) + E \cdot O(\log V)$ $= O((V + E) \log V) = O(E \log V)$
- *Standard choice for most applications.*



The Shortest-Path Tree

Dijkstra doesn't just compute distances — it builds a **shortest-path tree** (SPT).



The Shortest-Path Tree

Dijkstra doesn't just compute distances — it builds a **shortest-path tree** (SPT).

The parent pointers form a tree rooted at s where:

- Each vertex v has exactly one parent (except s).
- The path from s to v following parent pointers is the shortest path.

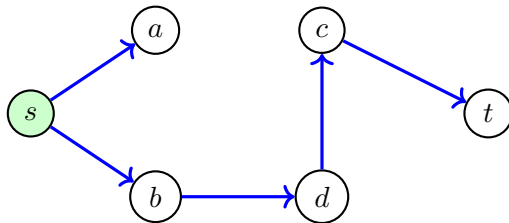


The Shortest-Path Tree

Dijkstra doesn't just compute distances — it builds a **shortest-path tree** (SPT).

The parent pointers form a tree rooted at s where:

- Each vertex v has exactly one parent (except s).
- The path from s to v following parent pointers is the shortest path.



Shortest-path tree from s

From Dijkstra to Bellman-Ford

Dijkstra's Algorithm:

- Finds shortest paths efficiently.
- Time: $O(E \log V)$ with binary heap.
- **Limitation:** Requires *non-negative weights*.



From Dijkstra to Bellman-Ford

Dijkstra's Algorithm:

- Finds shortest paths efficiently.
- Time: $O(E \log V)$ with binary heap.
- **Limitation:** Requires *non-negative weights*.

A more general problem:

- What if edges have *negative weights*?
- Need: **Bellman-Ford Algorithm**



From Dijkstra to Bellman-Ford

Dijkstra's Algorithm:

- Finds shortest paths efficiently.
- Time: $O(E \log V)$ with binary heap.
- **Limitation:** Requires *non-negative weights*.

A more general problem:

- What if edges have *negative weights*?
- Need: **Bellman-Ford Algorithm**

Real-world applications

Currency arbitrage detection, network routing with penalties, arbitrage in financial markets, finding optimal paths with costs and rewards.



The Single-Source Shortest Path with Negative Weights

Given: A directed graph $G = (V, E, W)$ with possibly negative edge weights, and a source vertex $s \in V$.



The Single-Source Shortest Path with Negative Weights

Given: A directed graph $G = (V, E, W)$ with possibly negative edge weights, and a source vertex $s \in V$.

Goal: Find the shortest path from s to every other vertex $v \in V$, or detect if a negative-weight cycle exists.



The Single-Source Shortest Path with Negative Weights

Given: A directed graph $G = (V, E, W)$ with possibly negative edge weights, and a source vertex $s \in V$.

Goal: Find the shortest path from s to every other vertex $v \in V$, or detect if a negative-weight cycle exists.

Negative-weight cycle: A cycle whose sum of edge weights is negative.

- If reachable from s , no shortest path exists (can loop infinitely to decrease distance).



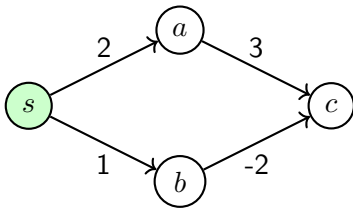
The Single-Source Shortest Path with Negative Weights

Given: A directed graph $G = (V, E, W)$ with possibly negative edge weights, and a source vertex $s \in V$.

Goal: Find the shortest path from s to every other vertex $v \in V$, or detect if a negative-weight cycle exists.

Negative-weight cycle: A cycle whose sum of edge weights is negative.

- If reachable from s , no shortest path exists (can loop infinitely to decrease distance).



Negative edge: $b \rightarrow c$ with weight -2



Why Dijkstra Fails with Negative Weights

Recall: Dijkstra assumes once a vertex is extracted, its distance is final.



Why Dijkstra Fails with Negative Weights

Recall: Dijkstra assumes once a vertex is extracted, its distance is final.

Problem: Negative edges can create shorter paths discovered later.



Why Dijkstra Fails with Negative Weights

Recall: Dijkstra assumes once a vertex is extracted, its distance is final.

Problem: Negative edges can create shorter paths discovered later.

Solution: Bellman-Ford relaxes *all edges repeatedly*, allowing later discoveries to correct earlier estimates.



Bellman-Ford Algorithm: The Main Idea

Key insight: In any shortest path, we visit at most $|V| - 1$ edges.



Bellman-Ford Algorithm: The Main Idea

Key insight: In any shortest path, we visit at most $|V| - 1$ edges.

Strategy:

- ➊ Start with $d[s] = 0$ and $d[v] = \infty$ for all $v \neq s$.
- ➋ Repeat $|V| - 1$ times:
 - **Relax** every edge (u, v) in the graph.
 - If $d[u] + w(u, v) < d[v]$, update $d[v] \leftarrow d[u] + w(u, v)$.
- ➌ After $|V| - 1$ iterations, all distances are correct (if no negative cycles).
- ➍ Check for negative cycles: try relaxing all edges once more.
If any distance improves, a negative cycle exists.



Bellman-Ford Algorithm: The Main Idea

Key insight: In any shortest path, we visit at most $|V| - 1$ edges.

Strategy:

- ➊ Start with $d[s] = 0$ and $d[v] = \infty$ for all $v \neq s$.
- ➋ Repeat $|V| - 1$ times:
 - **Relax** every edge (u, v) in the graph.
 - If $d[u] + w(u, v) < d[v]$, update $d[v] \leftarrow d[u] + w(u, v)$.
- ➌ After $|V| - 1$ iterations, all distances are correct (if no negative cycles).
- ➍ Check for negative cycles: try relaxing all edges once more.
If any distance improves, a negative cycle exists.

Why $|V| - 1$ iterations?

Any simple path has at most $|V| - 1$ edges. After k iterations, we've correctly computed all shortest paths using $\leq k$ edges.

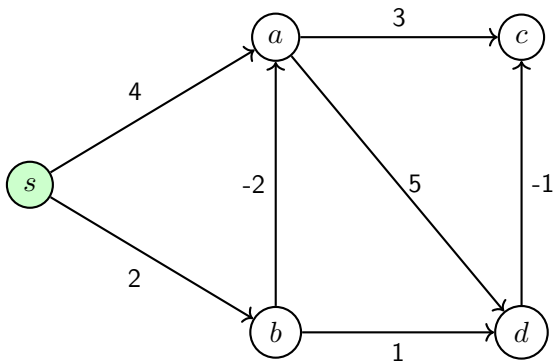


Bellman-Ford Algorithm: Pseudocode

```
procedure Bellman-Ford( $G = (V, E)$ ,  $s$ ,  $w : E \rightarrow \mathbb{R}$ )  
  for each vertex  $v \in V$  do  
     $d[v] \leftarrow \infty$ ;    $\text{parent}[v] \leftarrow \text{null}$   
   $d[s] \leftarrow 0$   
  
  for  $i = 1$  to  $|V| - 1$  do  
    for each edge  $(u, v) \in E$  do  
      if  $d[u] + w(u, v) < d[v]$  then  
         $d[v] \leftarrow d[u] + w(u, v)$   
         $\text{parent}[v] \leftarrow u$   
  
  for each edge  $(u, v) \in E$  do  
    if  $d[u] + w(u, v) < d[v]$  then  
      return "negative cycle detected"  
  
  return  $d$ , parent
```



Bellman-Ford Example: Initial Graph

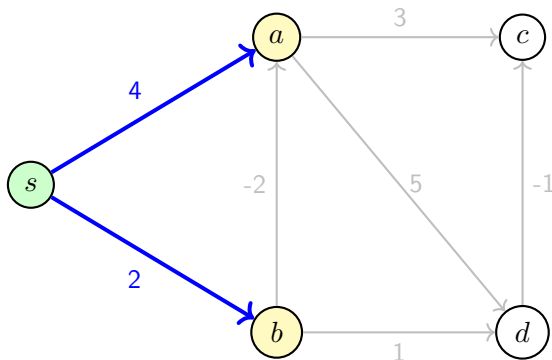


Initial distances: $d[s] = 0$, all others ∞ .

Note: Edges $b \rightarrow a$ and $d \rightarrow c$ have negative weights.



Iteration 1: Relax All Edges

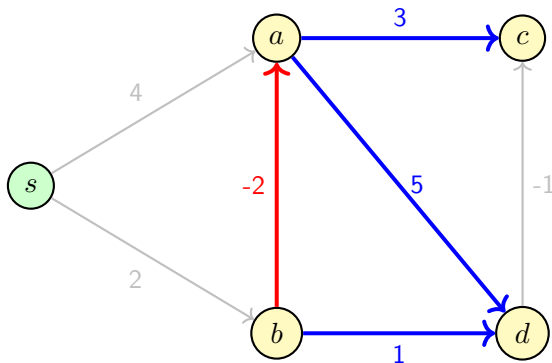


Vertex	<i>s</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
$d[v]$	0	4	2	∞	∞

Relaxed edges: $s \rightarrow a$ (set $d[a] = 4$), $s \rightarrow b$ (set $d[b] = 2$).

Other edges don't improve distances yet (source vertices still at ∞).

Iteration 2: Relax All Edges Again

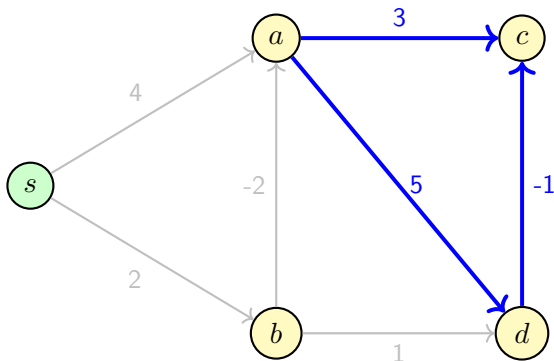


Vertex	s	a	b	c	d
$d[v]$	0	0	2	7	3

Key updates:

$$d[a] = d[b] + w(b, a) = 2 + (-2) = 0 \text{ (improved from 4)}$$

Iteration 3: Continue Relaxing

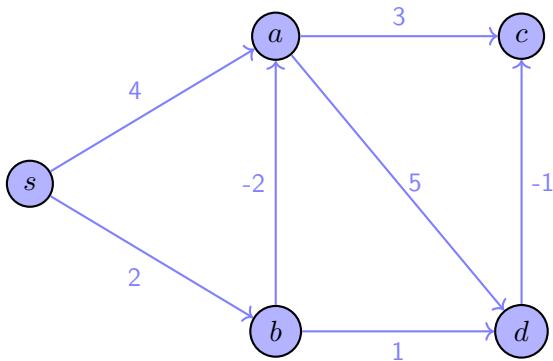


Vertex	s	a	b	c	d
$d[v]$	0	0	2	2	3

Key update:

$$d[c] = d[d] + w(d, c) = 3 + (-1) = 2 \text{ (improved from 3)}$$

Iteration 4: Final Iteration (if $|V| = 5$)



Vertex	s	a	b	c	d
$d[v]$	0	0	2	2	3

No distances changed in this iteration.

Algorithm terminates: All shortest distances found!

Negative Cycle Detection

After $|V| - 1$ iterations, try relaxing all edges **one more time**.



Negative Cycle Detection

After $|V| - 1$ iterations, try relaxing all edges **one more time**.

If any distance improves: A negative cycle exists!



Negative Cycle Detection

After $|V| - 1$ iterations, try relaxing all edges **one more time**.

If any distance improves: A negative cycle exists!

Why?

- After $|V| - 1$ iterations, all simple paths have been explored.
- If we can still improve a distance, we're using $\geq |V|$ edges.
- This means we're revisiting vertices $\Rightarrow$ a cycle exists.
- Since the distance decreases, the cycle has negative total weight.



Negative Cycle Detection

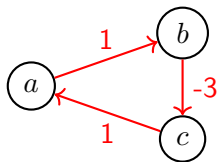
After $|V| - 1$ iterations, try relaxing all edges **one more time**.

If any distance improves: A negative cycle exists!

Why?

- After $|V| - 1$ iterations, all simple paths have been explored.
- If we can still improve a distance, we're using $\geq |V|$ edges.
- This means we're revisiting vertices $\Rightarrow$ a cycle exists.
- Since the distance decreases, the cycle has negative total weight.

Example with negative cycle:



Correctness of Bellman-Ford

Theorem: After k iterations of Bellman-Ford, $d[v]$ equals the shortest path distance from s to v using at most k edges.



Correctness of Bellman-Ford

Theorem: After k iterations of Bellman-Ford, $d[v]$ equals the shortest path distance from s to v using at most k edges.

Proof (by induction on k):



Correctness of Bellman-Ford

Theorem: After k iterations of Bellman-Ford, $d[v]$ equals the shortest path distance from s to v using at most k edges.

Proof (by induction on k):

- **Base case ($k = 0$):** $d[s] = 0$, all others ∞ . Correct.



Correctness of Bellman-Ford

Theorem: After k iterations of Bellman-Ford, $d[v]$ equals the shortest path distance from s to v using at most k edges.

Proof (by induction on k):

- **Base case ($k = 0$):** $d[s] = 0$, all others ∞ . Correct.
- **Inductive step:** Assume true for $k - 1$. Consider any shortest path P to v using $\leq k$ edges.
 - Let $P = s \rightsquigarrow u \rightarrow v$ where the last edge is (u, v) .
 - Path to u uses $\leq k - 1$ edges, so by induction, $d[u]$ is correct after iteration $k - 1$.
 - In iteration k , we relax edge (u, v) : $d[v] \leftarrow \min(d[v], d[u] + w(u, v))$.
 - Thus $d[v]$ becomes the shortest distance using $\leq k$ edges.



Correctness of Bellman-Ford

Theorem: After k iterations of Bellman-Ford, $d[v]$ equals the shortest path distance from s to v using at most k edges.

Proof (by induction on k):

- **Base case ($k = 0$):** $d[s] = 0$, all others ∞ . Correct.
- **Inductive step:** Assume true for $k - 1$. Consider any shortest path P to v using $\leq k$ edges.
 - Let $P = s \rightsquigarrow u \rightarrow v$ where the last edge is (u, v) .
 - Path to u uses $\leq k - 1$ edges, so by induction, $d[u]$ is correct after iteration $k - 1$.
 - In iteration k , we relax edge (u, v) : $d[v] \leftarrow \min(d[v], d[u] + w(u, v))$.
 - Thus $d[v]$ becomes the shortest distance using $\leq k$ edges.

Conclusion: After $|V| - 1$ iterations, all shortest simple paths (which have $\leq |V| - 1$ edges) are correctly computed.



Time Complexity of Bellman-Ford

Analysis:



Time Complexity of Bellman-Ford

Analysis:

- **Initialization:** $O(V)$ to set all distances to ∞ and $d[s] = 0$.



Time Complexity of Bellman-Ford

Analysis:

- **Initialization:** $O(V)$ to set all distances to ∞ and $d[s] = 0$.
- **Main loop:** $|V| - 1$ iterations.
 - Each iteration relaxes all $|E|$ edges.
 - Relaxation per edge: $O(1)$.
 - Total per iteration: $O(E)$.
- Total for main loop: $O(V \cdot E) = O(VE)$.



Time Complexity of Bellman-Ford

Analysis:

- **Initialization:** $O(V)$ to set all distances to ∞ and $d[s] = 0$.
- **Main loop:** $|V| - 1$ iterations.
 - Each iteration relaxes all $|E|$ edges.
 - Relaxation per edge: $O(1)$.
 - Total per iteration: $O(E)$.
- Total for main loop: $O(V \cdot E) = O(VE)$.
- **Negative cycle check:** One more pass over all $|E|$ edges: $O(E)$.



Time Complexity of Bellman-Ford

Analysis:

- **Initialization:** $O(V)$ to set all distances to ∞ and $d[s] = 0$.
- **Main loop:** $|V| - 1$ iterations.
 - Each iteration relaxes all $|E|$ edges.
 - Relaxation per edge: $O(1)$.
 - Total per iteration: $O(E)$.
- Total for main loop: $O(V \cdot E) = O(VE)$.
- **Negative cycle check:** One more pass over all $|E|$ edges: $O(E)$.

Overall time complexity: $O(VE)$.



Time Complexity of Bellman-Ford

Analysis:

- **Initialization:** $O(V)$ to set all distances to ∞ and $d[s] = 0$.
- **Main loop:** $|V| - 1$ iterations.
 - Each iteration relaxes all $|E|$ edges.
 - Relaxation per edge: $O(1)$.
 - Total per iteration: $O(E)$.
- Total for main loop: $O(V \cdot E) = O(VE)$.
- **Negative cycle check:** One more pass over all $|E|$ edges: $O(E)$.

Overall time complexity: $O(VE)$.

Comparison with Dijkstra:

- Dijkstra: $O(E \log V)$ with binary heap (but requires non-negative weights).
- Bellman-Ford: $O(VE)$ (slower, but handles negative weights).



Time Complexity of Bellman-Ford

Analysis:

- **Initialization:** $O(V)$ to set all distances to ∞ and $d[s] = 0$.
- **Main loop:** $|V| - 1$ iterations.
 - Each iteration relaxes all $|E|$ edges.
 - Relaxation per edge: $O(1)$.
 - Total per iteration: $O(E)$.
- Total for main loop: $O(V \cdot E) = O(VE)$.
- **Negative cycle check:** One more pass over all $|E|$ edges: $O(E)$.

Overall time complexity: $O(VE)$.

Comparison with Dijkstra:

- Dijkstra: $O(E \log V)$ with binary heap (but requires non-negative weights).
- Bellman-Ford: $O(VE)$ (slower, but handles negative weights).

For sparse graphs ($E \approx V$): Bellman-Ford is $O(V^2)$.

For dense graphs ($E \approx V^2$): Bellman-Ford is $O(V^3)$.



The Shortest Path Tree (with Negative Weights)

Like Dijkstra, Bellman-Ford constructs a **shortest-path tree** using parent pointers.



The Shortest Path Tree (with Negative Weights)

Like Dijkstra, Bellman-Ford constructs a **shortest-path tree** using parent pointers.

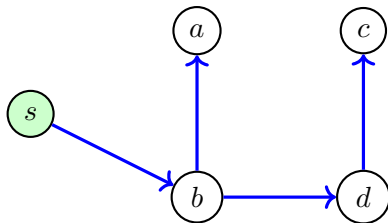
Difference: The tree may change during iterations as better paths are found.



The Shortest Path Tree (with Negative Weights)

Like Dijkstra, Bellman-Ford constructs a **shortest-path tree** using parent pointers.

Difference: The tree may change during iterations as better paths are found.

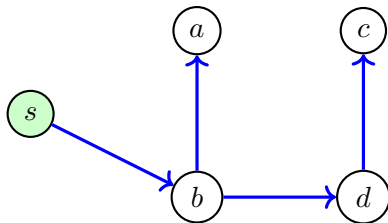


Shortest-path tree from s (including negative edges)

The Shortest Path Tree (with Negative Weights)

Like Dijkstra, Bellman-Ford constructs a **shortest-path tree** using parent pointers.

Difference: The tree may change during iterations as better paths are found.



Shortest-path tree from s (including negative edges)

Note: If a negative cycle is detected, no valid shortest-path tree exists for vertices reachable via the cycle.



The Shortest Path Algorithm Landscape

